

# Python - Continue Statement

## Python continue Statement

Python **continue statement** is used to skip the execution of the program block and returns the control to the beginning of the current **loop** to start the next iteration. When encountered, the loop starts next iteration without executing the remaining statements in the current iteration.

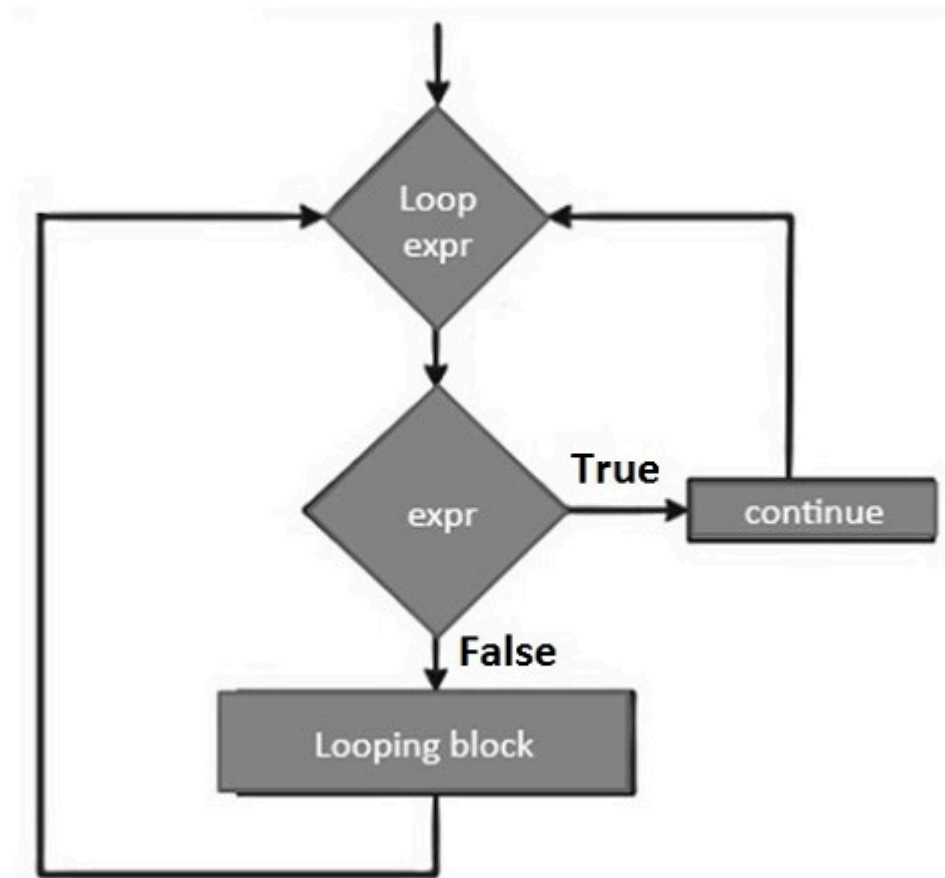
The **continue statement** is just the opposite to that of **break**. It skips the remaining statements in the current loop and starts the next iteration.

## Syntax of continue Statement

```
looping statement:  
    condition check:  
        continue
```

## Flow Diagram of continue Statement

The flow diagram of the continue statement looks like this –



## Python continue Statement with for Loop

In **Python**, the **continue** statement is allowed to be used with a **for loop**. Inside the for loop, you should include an **if statement** to check for a specific condition. If the condition becomes **TRUE**, the **continue** statement will skip the current iteration and proceed with the next iteration of the loop.

### Example

Let's see an example to understand how the **continue statement** works in for loop.

```

for letter in 'Python':
    if letter == 'h':
        continue
    print ('Current Letter :', letter)
print ("Good bye!")

```

When the above code is executed, it produces the following **output** –

```
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
Good bye!
```

## Python continue Statement with while Loop

Python **continue statement** is used with 'for' loops as well as 'while' loops to skip the execution of the current iteration and transfer the program's control to the next iteration.

### Example: Checking Prime Factors

Following code uses continue to find the prime factors of a given number. To find prime factors, we need to successively divide the given number starting with 2, increment the divisor and continue the same process till the input reduces to 1.



```
num = 60
print ("Prime factors for: ", num)
d=2
while num > 1:
    if num%d==0:
        print (d)
        num=num/d
        continue
    d=d+1
```

On executing, this code will produce the following **output** –

```
Prime factors for: 60
2
2
3
5
```

Assign different value (say 75) to num in the above program and test the result for its prime factors.

Prime factors for: 75

3

5

5

## TOP TUTORIALS

[Python Tutorial](#)

[Java Tutorial](#)

[C++ Tutorial](#)

[C Programming Tutorial](#)

[C# Tutorial](#)

[PHP Tutorial](#)

[R Tutorial](#)

[HTML Tutorial](#)

[CSS Tutorial](#)

[JavaScript Tutorial](#)

[SQL Tutorial](#)

## TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)

[Amazon Web Services Tutorial](#)

[Microsoft Azure Tutorial](#)



Chapters ▾

Categories ≡

[Linux Training Tutorial](#)

[Docker Tutorial](#)

[Kubernetes Tutorial](#)

[DSA Tutorial](#)

[Spring Boot Tutorial](#)

[SDLC Tutorial](#)

[Unix Tutorial](#)

## CERTIFICATIONS

[Business Analytics Certification](#)

Java & Spring Boot Advanced Certification  
Data Science Advanced Certification  
Cloud Computing And DevOps  
Advanced Certification In Business Analytics  
Artificial Intelligence And Machine Learning  
DevOps Certification  
Game Development Certification  
Front-End Developer Certification  
AWS Certification Training  
Python Programming Certification

## COMPILERS & EDITORS

Online Java Compiler  
Online Python Compiler  
Online Go Compiler  
Online C Compiler  
Online C++ Compiler  
Online C# Compiler  
Online PHP Compiler  
Online MATLAB Compiler  
Online Bash Terminal  
Online SQL Compiler  
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |  
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.